CareerPilot Project Pipelines

End-to-end pipeline map for the Placement Friend / CareerPilot AI monorepo. The project has two job ingestion paths: the newer CareerPilot ATS sync path that writes to jobs and job_matches, and the older worker scraper path that writes to drives and alerts_sent.

Student/User	Next.js Web App	NestJS API	PostgreSQL/Supabase
Worker Scheduler	Scraper/ATS Adapters	AI Providers	Notifications/Dashboard

- apps/web owns authenticated UI, dashboard screens, and proxy API routes.
- apps/api owns resume parsing, ATS sync, semantic search, match scoring, companies, and agent chat.
- apps/worker owns scheduled scraping, page/ATS extraction, drive ingestion, and notification dispatch.
- packages/domain owns shared opportunity and student eligibility rules.
- infra owns schema, CareerPilot migration, seed data, and database initialization.

Pipeline 1: Current CareerPilot ATS Job Sync

Triggered from the web app through the NestJS API. It fetches jobs from known ATS providers, normalizes them, optionally embeds them, upserts them into jobs, and auto-scores matches for the user.

Web Sync Button	POST /api/careerpilot/sync	POST /api/worker/sync	SyncService.syncAll
Load candidate profile	Load student row	Load ATS companies	Filter by CGPA/branch
Greenhouse	Lever	Ashby	Workday / SmartRecruiters
Normalize jobs	Hard eligibility pre-check	Generate Gemini embedding	Upsert jobs
Write sync_logs	Update company sync_status	Auto-match per job	Upsert job_matches

Entry files	apps/web/app/api/careerpilot/sync/route.ts, apps/api/src/sync/sync.controller.ts
Core service	apps/api/src/sync/sync.service.ts
Destination tables	jobs, job_matches, sync_logs, companies
External systems	Greenhouse, Lever, Ashby, Workday, SmartRecruiters, Gemini embeddings
Key behavior	Skips ineligible companies/jobs, times out slow companies, logs failures, preserves existing embeddings when job content is unchanged.

Pipeline 2: Worker Scraper, Drives, and Notifications

A scheduled worker cycle scrapes career pages. It can discover missing URLs, validate pages, detect ATS providers, fall back to AI/regex extraction, save deduplicated drives, then dispatch dashboard and mock email notifications.

Scheduler interval	executePipeline	Resolve active student	Fetch active companies
Missing URL?	Discover via search	Validate URL	Save or flag url_missing
Detect redirect	Scrape page	Detect login wall	Detect ATS provider
ATS adapter path	AI/regex fallback path	Raw opportunities	Baseline filters
Branch filter	Generate dedupe_key	Insert drives	Run eligibility matching
dispatchNotifications	Check duplicates	Insert alerts_sent	Append mock email log

Entry files	apps/worker/src/scheduler.ts, apps/worker/src/index.ts
Extraction files	apps/worker/src/scraper.ts, apps/worker/src/agent.ts, apps/worker/src/ats/*
Notification file	apps/worker/src/notifier.ts
Destination tables	drives, alerts_sent, system_state, companies
Key behavior	Uses company status/failure counters, domain rate limiting, dedupe_key conflict protection, and channel preferences per tracked company.

Resume Parsing and Candidate Profile Pipeline

Upload PDF/DOCX	Extract text	Groq JSON schema parser	Gemini parser fallback
Heuristic fallback	Normalize profile	Generate profile embedding	Upsert candidate_profiles
Delete old job_matches	Profile ready	Search/match can use vectors	Agent can read resume

Entry files	apps/web/app/api/careerpilot/resume/route.ts, apps/api/src/resume/resume.controller.ts
Core service	apps/api/src/resume/resume.service.ts
Destination table	candidate_profiles
External systems	pdf-parse, mammoth, Groq, Gemini embeddings
Key behavior	Sanitizes null bytes, normalizes skills/education/experience, clears stale match results after profile changes.

Search, Match, and CareerPilot Agent Pipeline

User asks CareerPilot	Load/create conversation	Groq/Gemini planner	Choose tool or answer
read_resume	search_jobs	compute_match	get_skill_gap
Vector search if embeddings exist	Keyword fallback search	Hard requirement checks	LLM/heuristic scoring
Save conversation	Return answer to web app	Dashboard reads matches	User applies externally

Agent service	apps/api/src/agent/agent.service.ts
Job matching service	apps/api/src/jobs/jobs.service.ts
Destination tables	conversations, job_matches
Search data	jobs joined with companies, with optional pgvector similarity
Hard checks	Experience, degree, branch, graduation year, location, seniority.

Database Relationship Summary

students	student_company_targets	companies	drives
alerts_sent	jobs	candidate_profiles	job_matches
sync_logs	conversations	system_state	student_notification_preferences

- students -> student_company_targets -> companies defines what each student tracks.
- companies -> drives -> alerts_sent belongs to the worker placement-drive notification flow.
- companies -> jobs -> job_matches belongs to the CareerPilot ATS sync and semantic match flow.
- candidate_profiles links a user resume to job_matches and agent personalization.
- sync_logs records per-company ATS sync results for observability.
- system_state stores runtime markers such as active_student_id and last_notifier_run.

Where The Two Job Models Split

drives model	Created by the worker scraper. Used for placement-style opportunities, branch/CGPA eligibility, alerts_sent, dashboard/email notifications.
jobs model	Created by the CareerPilot ATS sync. Used for normalized ATS jobs, embeddings, semantic search, resume-aware scoring, and job_matches.
Shared inputs	companies, students, student_company_targets, candidate_profiles when user-specific sync is requested.
Practical implication	The project is not a single straight pipeline yet. It has two ingestion systems that should eventually be unified or bridged if the product wants one canonical opportunity model.

Recommended future consolidation: make jobs the canonical role table, keep drives as campus-specific events only if needed, and route notifications from job_matches or a unified opportunity_matches table.